Команды Git для практики

Шпаргалка помогает выбрать команду и проверить, в какой папке и ветке её выполнять. Все команды вводим в терминале: Git Bash на Windows или Терминал на macOS.

Обозначения. <URL>, <URL-оригинала> и <HASH> целиком заменяем своим адресом или хешем, без угловых скобок. main — основная ветка. improve-topic — рабочая ветка из практики.

Адреса. В клоне своего fork origin ведёт в ваш fork, upstream — в оригинал автора. Проверьте через git remote -v. Fork и PR создаём в браузере на GitHub.

Команда Git	Что делает и где выполнять
`git --version`	**Действие:** Показывает установленную версию Git. **Где:** В терминале из любой папки.
`git config --global user.name "Имя Фамилия"`	**Действие:** Задаёт имя автора новых коммитов по умолчанию. Это не вход в GitHub. **Где:** Из любой папки. Подставьте своё имя. Без --global настройка действует только в текущем репозитории.
`git config --global user.email "your-email@example.com"`	**Действие:** Задаёт email автора новых коммитов. Используйте свой подтверждённый email GitHub или адрес noreply из его настроек. **Где:** Из любой папки. Без --global настройка действует только в текущем репозитории.
`git init -b main`	**Действие:** Создаёт локальный Git-репозиторий и задаёт имя начальной ветки main. Удалённый репозиторий не создаёт. **Где:** В корневой папке нового проекта. Первый коммит нужно сделать отдельно.
`git clone <URL>`	**Действие:** Клонирует существующий репозиторий в новую папку и обычно сохраняет его адрес под именем origin. **Где:** В родительской папке для будущего проекта. Затем перейдите в созданную папку. Для работы с fork клонируйте свой fork.
`git status`	**Действие:** Показывает текущую ветку, подготовленные и неподготовленные правки, новые файлы и состояние незавершённой операции. **Где:** В папке нужного репозитория. Чистое дерево не означает, что коммиты уже отправлены на GitHub.
`git status --short`	**Действие:** Показывает состояние файлов в кратком виде. ?? означает новый неотслеживаемый файл. **Где:** В папке репозитория. Для полного пояснения используйте git status.
`git remote -v`	**Действие:** Показывает сохранённые имена удалённых подключений и их URL. **Где:** В папке репозитория. Проверьте адреса перед обменом изменениями.

Команда Git	Что делает и где выполнять
`git remote add origin <URL>`	**Действие:** Сохраняет адрес удалённого репозитория под именем origin. Файлы и коммиты не передаёт. **Где:** В локальном репозитории, созданном через init. После clone origin обычно уже существует.
`git remote add upstream <URL-оригинала>`	**Действие:** Добавляет адрес исходного проекта автора под именем upstream. Само добавление адреса не получает изменения. **Где:** В локальном клоне своего fork. Обычно один раз при настройке. origin при этом ведёт в ваш fork.
`git remote set-url origin <URL>`	**Действие:** Меняет адрес уже существующего origin, например при переходе с HTTPS на SSH. **Где:** В папке репозитория. Сначала проверьте текущий адрес через git remote -v.
`git add README.md`	**Действие:** Подготавливает текущее содержимое указанного файла в индексе. Для нового файла также начинает отслеживание. **Где:** В корне репозитория, где лежит README.md. После новых правок повторите add, если хотите включить их в коммит.
`git add .`	**Действие:** Подготавливает изменения в текущей папке и её подпапках, включая новые файлы и удаления. Учитывает правила игнорирования. **Где:** В корне репозитория, если нужен весь проект. Сначала проверьте status, затем подготовленную разницу.
`git check-ignore -v scratch.txt`	**Действие:** Показывает, какое правило игнорирует указанный файл. Полезно для проверки .gitignore. **Где:** В корне проекта с scratch.txt. Отсутствие вывода может означать, что подходящего правила нет.
`git diff`	**Действие:** Показывает разницу между рабочими файлами и индексом: ещё не подготовленные правки. Новые неотслеживаемые файлы не показывает. **Где:** В папке репозитория, перед add или после дополнительных правок.
`git diff --staged` `git diff --cached`	**Действие:** Два равнозначных варианта. Показывают подготовленные изменения относительно последнего коммита текущей ветки. До первого коммита показывают всё подготовленное содержимое. **Где:** В папке репозитория после add, перед commit. Проверьте, что именно собираетесь сохранить.
`git diff HEAD`	**Действие:** Показывает разницу рабочих файлов с текущим коммитом, включая подготовленные и неподготовленные правки отслеживаемых файлов. **Где:** В репозитории, где уже есть коммит. В обычной работе HEAD через текущую ветку указывает на её последний коммит.

Команда Git	Что делает и где выполнять
`git restore --staged README.md`	**Действие:** Убирает подготовленные изменения файла из индекса, сохраняя правки в рабочем файле. **Где:** В корне репозитория, где уже существует первый коммит. Используйте именно --staged, чтобы изменить индекс.
`git commit -m "Добавлена памятка QA"`	**Действие:** Создаёт локальный коммит из подготовленного содержимого и продвигает текущую ветку. -m задаёт сообщение. **Где:** В репозитории после add и проверки diff --staged. Сначала убедитесь, что выбрана нужная ветка.
`git log --oneline -5`	**Действие:** Показывает пять последних коммитов в кратком виде: хеш и сообщение. **Где:** В папке репозитория. Если открылся просмотрщик, клавиша q закрывает его.
`git show HEAD`	**Действие:** Показывает сведения о текущем коммите и его изменения. Вместо HEAD можно указать известный хеш. **Где:** В репозитории, где уже есть коммит.
`git log --oneline --graph --all`	**Действие:** Показывает известную локально историю со связями между коммитами всех имеющихся ссылок. Сам не получает данные с GitHub. **Где:** В папке репозитория. Для свежей удалённой истории сначала выполните fetch нужного remote.
`git branch`	**Действие:** Показывает список локальных веток. Текущая обычно отмечена звёздочкой. **Где:** В папке репозитория.
`git branch --show-current`	**Действие:** Показывает имя текущей ветки. **Где:** В папке репозитория перед commit, merge, pull или cherry-pick.
`git switch -c improve-topic`	**Действие:** Создаёт ветку improve-topic от текущего положения и переключается в неё. **Где:** В нужном репозитории после проверки исходной ветки и версии. В практике: в папке клона fork до правок темы.
`git switch main`	**Действие:** Переключает в существующую локальную main. Незакоммиченные правки могут перейти вместе с переключением или помешать ему. **Где:** В папке репозитория. В учебном процессе сначала проверьте status и сохраните нужную работу коммитом.
`git merge improve-topic`	**Действие:** Включает изменения improve-topic в текущую ветку. Возможны fast-forward, коммит слияния или конфликт. Результат остаётся локальным. **Где:** В ветке ПОЛУЧАТЕЛЕ. Чтобы включить работу в main, сначала выполните git switch main. Начинайте с чистым рабочим деревом.

Команда Git	Что делает и где выполнять
`git fetch origin`	**Действие:** Получает коммиты и обновляет локальные сведения о ветках origin. Сам не объединяет их с текущей веткой. **Где:** В нужном репозитории, на любой ветке. origin/main может обновиться, а локальная main останется прежней.
`git fetch upstream`	**Действие:** Получает историю оригинала автора через заранее настроенный upstream. Рабочие файлы сам не обновляет. **Где:** В локальном клоне fork. Активная ветка не определяет, какие файлы обновит fetch: он их не объединяет.
`git branch -r`	**Действие:** Показывает известные локально ссылки на удалённые ветки, например origin/main и upstream/main. **Где:** В папке репозитория. Для обновления сведений сначала выполните fetch.
`git merge upstream/main`	**Действие:** Включает полученные изменения основной ветки оригинала в текущую локальную ветку. Может остановиться на конфликте. **Где:** В практике: в папке fork, в improve-topic, после git fetch upstream. Перед началом дерево должно быть чистым.
`git add README.md` `git commit -m "Согласована тема встречи"`	**Действие:** После ручного исправления README отмечает конфликт разрешённым и завершает обычный конфликтный merge. Выполняйте строки по очереди. **Где:** В той же ветке и папке, где остановился merge. Сначала удалите маркеры, сохраните и проверьте итоговый текст.
`git merge --abort`	**Действие:** Отменяет текущий незавершённый merge. Уже созданный коммит слияния не отменяет. **Где:** В том же репозитории, пока merge ещё не завершён. В нашем сценарии до merge рабочее дерево было чистым.
`git push -u origin main`	**Действие:** Публикует локальную main в origin и настраивает связь с origin/main. -u не создаёт remote с именем upstream. **Где:** В своём репозитории после первого коммита. В учебном сценарии на GitHub заранее создан пустой репозиторий, origin настроен.
`git push -u origin improve-topic`	**Действие:** Публикует рабочую ветку в origin и настраивает её связь с удалённой веткой. **Где:** В практике: в папке fork, после коммита в improve-topic. origin должен вести в ваш fork.
`git push`	**Действие:** В обычном настроенном сценарии отправляет коммиты текущей ветки в связанную удалённую ветку. Обновляет открытый PR этой ветки. **Где:** После настройки связи через push -u. Для исправления PR оставайтесь в его исходной ветке. Незакоммиченные правки не отправляет.

Команда Git	Что делает и где выполнять
`git pull --ff-only origin main`	**Действие:** Получает main из origin и обновляет текущую ветку только простым продвижением ссылки. При расхождении истории останавливается. **Где:** В нашем сценарии: после git switch main и с чистым деревом. origin/main уже должна содержать нужный результат, например принятый PR.
`git merge --ff-only upstream/main`	**Действие:** Обновляет текущую ветку до полученного upstream/main, только если возможен fast-forward. **Где:** Для синхронизации main fork: в папке fork сначала git switch main и git fetch upstream. Затем можно git push origin main.
`git cherry-pick <HASH>`	**Действие:** Применяет изменение выбранного коммита в текущей ветке, обычно создавая новый коммит. Исходный коммит остаётся на месте. **Где:** В ветке ПОЛУЧАТЕЛЕ, с чистым деревом. Сначала переключитесь в неё. Хеш возьмите из log, при необходимости получите коммит через fetch.
`git cherry-pick --continue`	**Действие:** Продолжает остановившийся cherry-pick после разрешения конфликта. **Где:** В той же ветке и папке. Сначала исправьте файлы, удалите маркеры, сохраните результат и выполните git add для исправленных файлов.
`git cherry-pick --abort`	**Действие:** Отменяет незавершённый cherry-pick и возвращает состояние до начала переноса. **Где:** В репозитории, где перенос ещё не завершён. Для обычного merge используется другая команда: git merge --abort.

Перед merge, pull и cherry-pick в учебном сценарии проверяйте git status. При конфликте сначала согласуйте и исправьте содержимое, затем завершайте именно ту операцию, которую начали.

Документация: git-scm.com/docs — справочник команд Git. Отдельные инструкции: git-scm.com/docs/git-cherry-pick, git-scm.com/docs/git-pull, git-scm.com/docs/git-restore.